PARALLEL COMPUTING LAB

25 Important Viva Questions with Answers

Covers the 9 practical experiments shown in the shared syllabus, plus CUDA/GPU basics.

OpenMP – Merge Sort Using Sections

1. What is OpenMP?

Answer: OpenMP is an API used to write parallel programs for shared-memory systems using compiler directives, library routines and environment variables.

2. What is merge sort?

Answer: Merge sort is a divide-and-conquer sorting algorithm. It divides an array, sorts the parts and merges them in sorted order.

3. What is the purpose of parallel sections?

Answer: It assigns independent blocks of code to different threads so they can execute concurrently.

OpenMP – Static Scheduling

4. What is loop scheduling in OpenMP?

Answer: Scheduling decides how loop iterations are divided among threads.

5. What does schedule(static,2) mean?

Answer: It assigns iterations in chunks of two to threads in a predetermined, round-robin manner.

6. What is the use of omp_get_thread_num()?

Answer: It returns the ID of the thread currently executing.

OpenMP – Fibonacci Using Tasks

7. What is the Fibonacci series?

Answer: It is a sequence where each term after the first two is the sum of the previous two: 0, 1, 1, 2, 3, 5, 8, ...

8. What is an OpenMP task?

Answer: A task is a unit of work that can be executed by any available thread in the current team.

9. What is the use of taskwait?

Answer: It makes the current task wait until its child tasks have completed.

OpenMP – Prime Numbers Using Parallel For

10. What is a prime number?

Answer: A whole number greater than 1 that has exactly two positive factors: 1 and itself.

11. What is the use of #pragma omp parallel for?

Answer: It creates a team of threads and distributes loop iterations among them.

12. What is a race condition?

Answer: It occurs when threads access shared data at the same time and the result depends on the order of access.

MPI – Send and Receive

13. What is MPI?

Answer: MPI stands for Message Passing Interface. It is a standard for communication between processes in parallel programs.

14. What are MPI_Send() and MPI_Recv() used for?

Answer: MPI_Send() sends a message and MPI_Recv() receives a message between processes.

15. What are MPI_Comm_rank() and MPI_Comm_size()?

Answer: MPI_Comm_rank() returns a process's rank; MPI_Comm_size() returns the number of processes in the communicator.

MPI – Deadlock and Avoidance

16. What is deadlock?

Answer: Deadlock is a situation where processes wait indefinitely for one another and communication cannot proceed.

17. How can deadlock be avoided?

Answer: Use a safe order of send and receive operations or suitable non-blocking communication such as MPI_Isend() and MPI_Irecv().

MPI – Broadcast

18. What is MPI_Bcast()?

Answer: It sends the same data from one root process to all processes in a communicator.

19. What is the root process?

Answer: The root is the process that supplies the data for a collective operation such as broadcast.

MPI – Scatter and Gather

20. What is MPI_Scatter()?

Answer: It distributes portions of data from the root process to all processes.

21. What is MPI_Gather()?

Answer: It collects data from all processes and places the combined data at the root process.

MPI – Reduce and Allreduce

22. What is MPI_Reduce()?

Answer: It combines values from all processes using an operation such as sum, minimum, maximum or product, and stores the result at the root.

23. What is MPI_Allreduce()?

Answer: It performs a reduction and makes the final result available to every process.

CUDA / GPU Programming – Basic Viva

24. What is CUDA?

Answer: CUDA stands for Compute Unified Device Architecture. It is NVIDIA's parallel computing platform and programming model for GPUs.

25. What is the difference between CPU and GPU?

Answer: A CPU has a smaller number of powerful general-purpose cores, while a GPU has many cores designed to handle large numbers of parallel operations.

Common Questions for All Programs

26. What is speedup?

Answer: Speedup is serial execution time divided by parallel execution time.

27. What is the difference between a process and a thread?

Answer: A process is an independent program in execution with its own memory space. A thread is an execution unit within a process; threads can share the process's memory.